

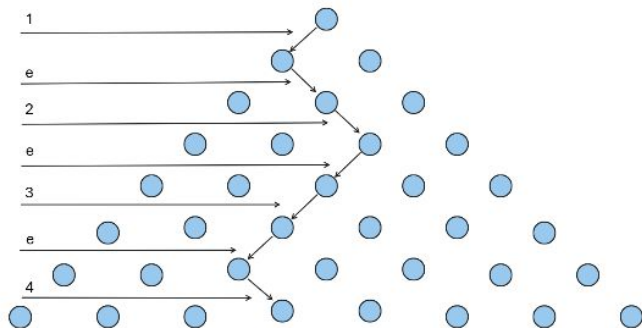
Irregular Beat Generator Algorithme

Strategie beschrijving

- Een algoritme (semi) gebaseerd op het spel plinko
- Elke stap is gelijk aan een noot.
- Er zijn zoveel stappen als er pulses zijn en een 50/50 kans berekening of er een stap naar links of rechts wordt gedaan.
- Een stap naar link is een noot, een stap naar rechts is een rust.
- De grootte v/d plinko wordt bepaald door de pulse (user input), het aantal noten bepaald het aantal toegestane stappen naar links (user input).
- Het uiteindelijke ritme is gebaseerd op het pad dat de bal aflegt niet de uiteindelijke uitkomst van waar de bal terecht komt.
- dit wordt 3 keer (minimaal) gedaan om 3 verschillende paden (ritmes) te generen (1x Kick, 1x Snare, 1x HH)
- Hier kunnen van te voren regels aan mee worden gegeven zoals de snare speelt nooit tegelijk met de kick, hier wordt dan rekening mee gehouden in het pad.
- Er is dus een volgorde waarin de paden worden gegenereerd (om rekening te kunnen houden met de voorgeprogrammeerde regels)
- Om wat meer ook een plinko idee mee te geven kan het uiteindelijke slot waar de bal terecht komt een parameter bepalen zoals een accent (kan dan dus ook op een rust vallen, dus geen accent)

Voorbeeld:

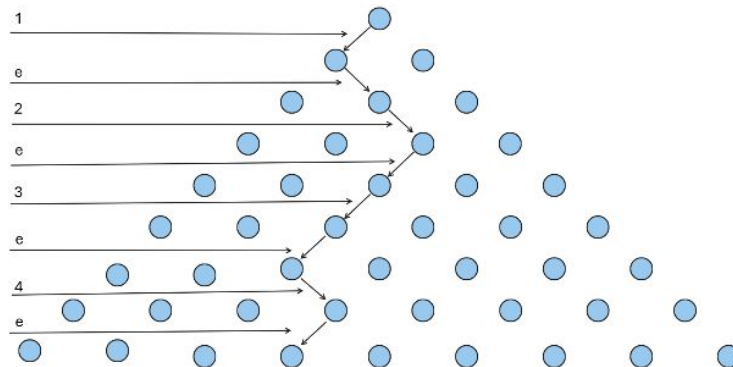
Voorbeeld in 7/8:



patroon = L R R L L L R

L = Beat
R = Rust

Voorbeeld in 4/4 :



patroon = L R R L L L R

L = Beat
R = Rust

Pro's & Con's

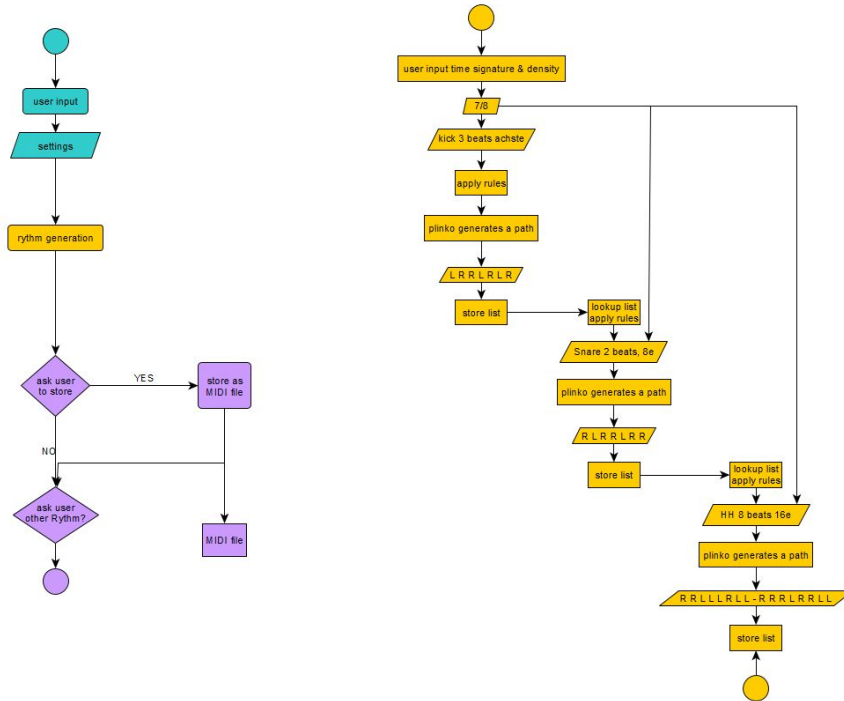
PRO

- Relatief simpel en overzichtelijk.
- Kunnen regels worden meegegeven die kunnen leiden tot een gecontroleerde random uitkomst.
- Elk instrument kan afzonderlijk worden gegenereerd, maar ook rekening houden met eerder gegenereerde instrumenten.

CON

- Te veel regels kunnen het complex maken.
- Kans op herhaling door beperkte opties wanneer er te veel regels worden mee gegeven.
- Lastig variëren (binnen 1 instrument) tussen verschillende noot waarde).
- Goede parameters (regels) moeten van te voren worden bedacht en geprogrammeerd om een interessante uitkomst te genereren.

Flowchart



rules:
rules can be things like a kick should always start on 1
or the first kick (out of 4 measures) should start on 1
snare can never play with kick

probability can be added like the probability that u get
a snare before a kick is low but after is high

Pseudo Code

```
C:\Users> brach_jm042he > Downloads > HBU > jar_2 > CSD > hoolidopdracht > ps pseudoCode.js > ...
1 //user inputs:
2 //pulse/time signature
3 //amount of notes
4 //bpm
5
6
7
8 let pulse = /*user input between [] & []*/;
9 let note = /*user input between [] & [(pulse)*/]; //example: if pulse is 7, notes is 1 to 7
10
11 //kick rules
12 apply kick rules {
13   step 1 is always 1
14 }
15
16 (amount of steps == (pulse));
17 (amount of 1 is (notes));
18 (amount of # steps is (pulse)-(notes));
19
20 flip a coin (50% L/50% R);
21 if (L){
22   write note
23 }
24 if (R){
25   write rest
26 }
27
28 does this violate a rule?
29 YES -> DON'T allow this direction
30 NO -> allow it
31 make kick list []
32
33 //snare rules
34 apply snare rules {
35   look up list kick
36   never at the same time as kick
37 }
38
39
40 (amount of steps == (pulse));
41 (amount of 1 steps is (notes)); //amount of snare notes/L steps????
42 (amount of # steps is (pulse)-(notes));
43
44 flip a coin (50% L/50% R);
45 if (L){
46   write note
47 }
48 if (R){
49   write rest
50 }
51
52 does this violate a rule?
53 YES -> DON'T allow this direction
54 NO -> allow it
55 make snare list []
56
```

```
57 //snare rules
58 apply snare rules {
59   look up list kick
60   look up list snare
61   (HNotes = Notes * 2)
62 }
63
64 (amount of steps == (pulse));
65 (amount of 1 is (notes));
66 (amount of # steps is (pulse)-(notes));
67
68 flip a coin (50% L/50% R);
69 if (L){
70   write note
71 }
72 if (R){
73   write rest
74 }
75
76 does this violate a rule?
77 YES -> DON'T allow this direction
78 NO -> allow it
79 make snare list []
80
81
82
83 repeat 4 times (4 bars)
84
85
86
87 //save as midi
88 save as midi file
89 [yes/no]
90 if (yes){
91   save as midi file
92 }
93
94 else {
95   ask to run again [A]
96   go back to the begin [B]
97   if (A) {
98     run again
99   }
100   else {
101     back to begin
102   }
103 }
104
105
```